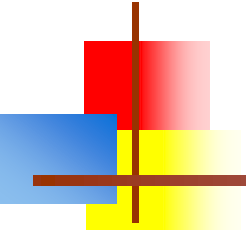
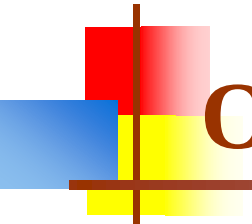


Formal Language and Complexity Theory



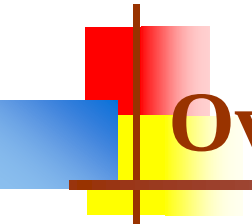
Chapter one: Introduction to the Theory of Computation





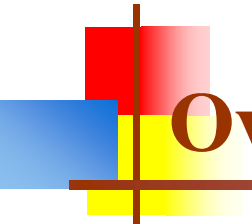
Outlines

- **Overview of languages : natural Vs formal**
- **Introduction to the Theory of Computation**
 - Mathematical Preliminaries and Notation
 - Sets and Subsets
 - Set operations
 - Languages
 - Grammars
 - Automata



Overview of languages : natural Vs formal

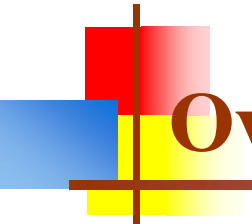
- Natural Languages
 - rules come after the language
 - evolve and develop
 - highly flexible
 - quite powerful
 - no special learning effort needed
 - Disadvantages
 - vague
 - imprecise
 - ambiguous
 - user and context dependent
 - Ex. Amharic, English, French, ...



Overview of languages: cont'd

■ Formal Languages

- developed with strict rules
 - predefined syntax and semantics
- precise
- unambiguous
 - can be processed by machines!
- Disadvantages
 - unfamiliar notation
 - initial learning effort
 - Ex. Programming languages: Pascal, C++, Java...



Overview of languages: cont'd

- Sentences: the basic building blocks of languages
- Sentence = Syntax + Semantics
- Grammar: the study of the structure of a sentence

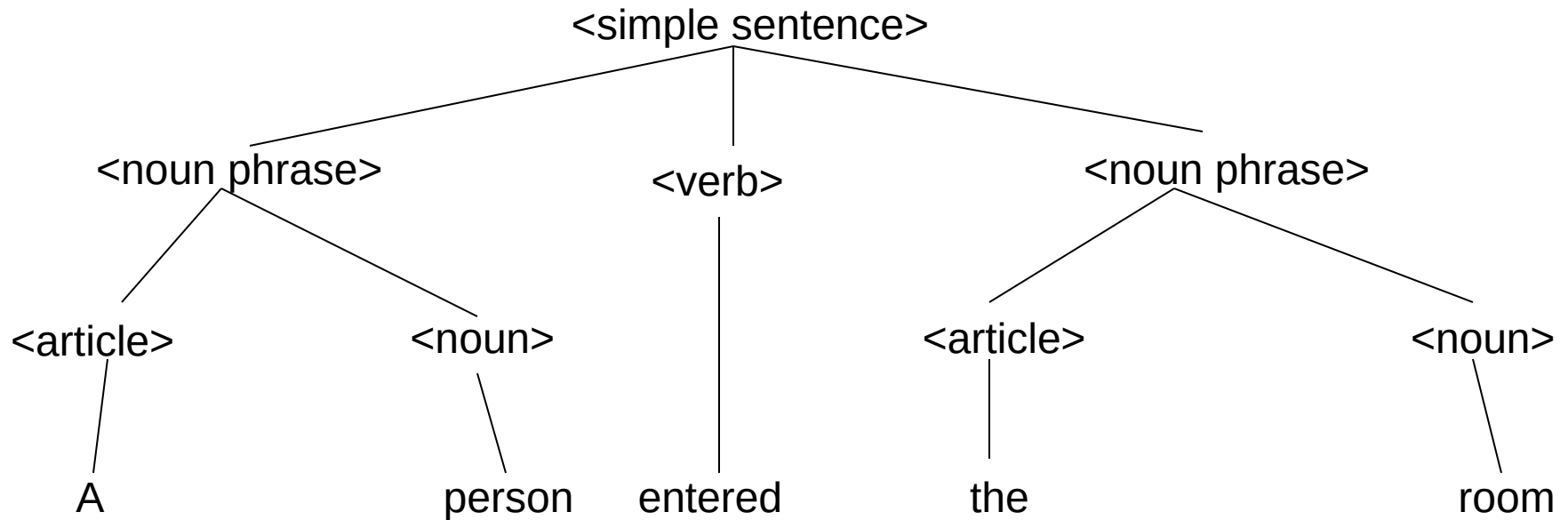
■ Ex:

$\langle \text{simple sentence} \rangle ::= \langle \text{noun phrase} \rangle \langle \text{verb} \rangle \langle \text{noun phrase} \rangle$

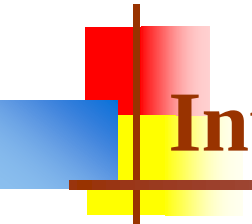
$\langle \text{noun phrase} \rangle ::= \langle \text{article} \rangle \langle \text{noun} \rangle$

→ A person entered the room

Overview of languages: cont'd

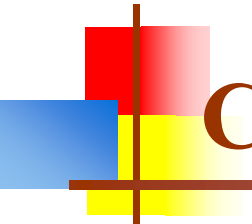


Derivation tree for the simple sentence: A person entered the room.



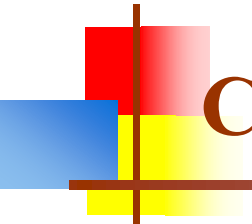
Introduction theory of computation

- The **theory of computation(TOC)** is a branch of **computer science** and **mathematics** that deals with what problems can be solved on a model of computation, using an algorithm, and how efficiently they can be solved.
- In order to perform a rigorous study of **computation**, **computer scientists** work with a mathematical abstraction of computers called a model of computation.
- There are several models in use, but the most commonly examined is the **Turing machine**.



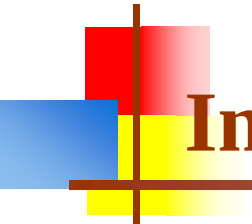
Cont'd...

- The **theory of computation** encompasses several *subfields*, including
 - **Automata theory**
 - **Computability theory**
 - **Complexity theory**



Cont'd...

- **Automata theory** deals with the study of abstract machines and their computational capabilities, while
- **Computability theory** deals with the study of what problems can be solved by computers and what problems cannot be solved by computers.
- **Complexity theory**, on the other hand, focuses on the resources (time and space) needed to solve problems.



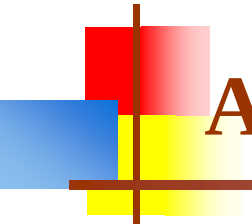
Introduction theory of computation

- **Automata theory** is the study of abstract computational devices.
- Automaton = an abstract computing device.
- These models play a role in several applied areas of computer science.
- One model, called the finite automaton, is used in **text processing, compilers, and hardware design.**
- Another model, called the **context-free grammar**, is used in programming languages and artificial intelligence.
- **Automata theory** is an excellent place to begin the study of the theory of computation.



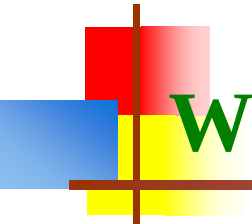
Applications of Theory of computation

- The **theory of computation** has various practical applications across different fields. Some of these **applications** include:
 - **Software Engineering**: The theory of computation is used to develop efficient mobile apps and software.
 - Algorithms built on computational theory principles help these applications run efficiently without consuming too much CPU power.
 - **Data Science**: It is used to create efficient algorithms for data analysis and predictions



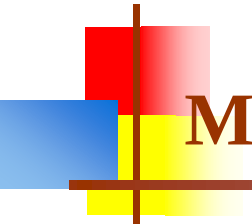
Applications of Theory of computation

- **Artificial Intelligence:** The theory of computation is critical for the construction of intelligent technology and the development of AI algorithms.
- **Cryptography:** It plays a vital role in cryptography, helping in the development of secure communication protocols and encryption algorithms.
- **Compiler Design:** It is important in the research and development of programming languages and in the design and construction of efficient compilers.
- The theory of computation is also applied in various practical systems such as **traffic lights, video games, and CPU controllers**



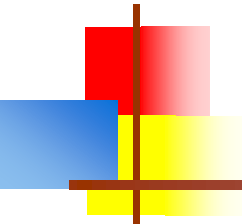
Why study theory?

- The **first answer** is that **theory provides concepts and principles** that help us understand the general nature of the discipline.
- A **second**, and perhaps not so obvious, answer is that the ideas we will discuss have some immediate and important applications.
- The **third answer** is one of which we hope to convince the reader. The subject matters intellectually stimulating and fun.



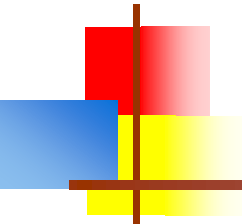
Mathematical Preliminaries and Notation

- **Sets**
- **Set Operations**
- The usual set operations are **union ($\cup$)**, **intersection ($\cap$)**, and **difference ($-$)** defined as
- **Relations and Functions**
- **Properties of Relations**
- **Graphs and Trees**
- **Proof techniques**



Sets

- A **set** is a well defined collection of objects, for example, the set of all students in a university.
- A **collection** of all books in a library also a set. The individual objects are called members or elements of the set.
- We use capital letters **A, B, C, . . .** for denoting sets, The small letters **a, b, c , . . .** are used to denote the elements of any set. .
- To indicate that **x** is an element of the set **A**, we write **$x \in A$** . The statement that **x** is not in **A** is written **$x \notin A$** .



Sets

We can denote sets using

1. By listing its elements

For example, the set of all positive integers divisible by 15 and less than 100 can be written as

$\{15, 30, 45, 60, 75, 90\}$.

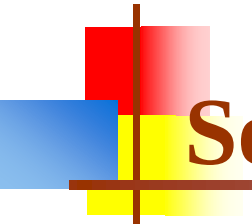
2. By describing the properties of the elements of the set.

- For example, the set $\{15, 30, 45, 60, 75, 90\}$ can be described as: $\{n: n \text{ is a positive integer divisible by 15 and less than 100}\}$. (The description of the property is called predicate. In this case the set is said to be implicitly specified.)

3.By recursion

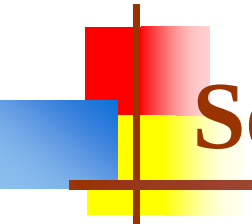
- We define the elements of the set by a computational rule for calculating the elements.
- For example. the set of all natural numbers leaving a remainder 1 when divided by 3 can be described as

$$\{a_n = | a_0 = 1, a_{n+1} = a_n + 3\}$$



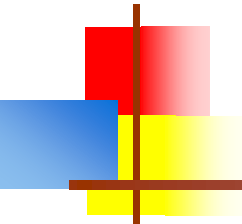
Set operations

- The usual set operations are
 - union ($\cup$), intersection ($\cap$), and difference ($-$) defined as
- If we have two sets $S1$ and $S2$, then union operation is defined by:-
 - $S1 \cup S2 = \{x: x \in S1, \text{ or } x \in S2\}$
- If we have two sets $S1$ and $S2$, then intersection operation is defined by:-
 - $S1 \cap S2 = \{x: x \in S1, \text{ and } x \in S2\}$
- If we have two sets $S1$ and $S2$, then difference operation is defined by:-
 - $S1 - S2 = \{x: x \in S1, \text{ or } x \notin S2\}$



Set operations

- Another basic operation is **complementation**.
- The complement of a set S , denoted by $\bar{S}$, which consists of all elements not in S .
- To make this meaningful, we need to know what the universal set U of all possible elements is. If U is specified, then $\bar{S} = \{x : x \in U, x \notin S\}$.

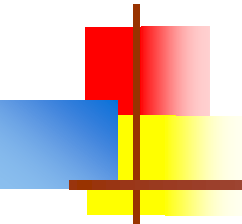


Set operations

- The set with no elements, called the empty set or the null set, is denoted by $\emptyset$.
- From the definition of a set, it is obvious that $S \cup \emptyset = S$, $S - \emptyset = S$, $S \cap \emptyset = \emptyset$, $\emptyset^c = U$, $\overline{\overline{S}} = S$. The following useful identities, known as **DeMorgan's laws**,

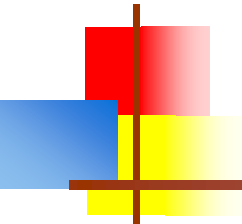
$$\overline{S_1 \cup S_2} = \overline{S_1} \cap \overline{S_2}, \quad \overline{S_1 \cap S_2} = \overline{S_1} \cup \overline{S_2}$$

are needed on several occasions



Set operations

- A set S_1 is said to be a **subset** of S if every element of S_1 is also an element of S . We write this as $S_1 \subseteq S$.
- We say that S_1 is a **proper subset** of S , if S contains at least one element which is not present in S_1 ; we write this as $S_1 \subset S$.
- If S_1 and S_2 are two sets and have no common element, that is, $S_1 \cap S_2 = \emptyset$, then the two sets S_1 and S_2 are said to be **disjoint sets**.



Set operations

- A set is said to be finite if it contains a finite number of elements; otherwise it is infinite.
- The size of a finite set is the number of elements in it; this is denoted by $|S|$.
- A given set normally has many subsets. The set of all subsets of a set S is called the **powerset** of S and is denoted by 2^S . Observe that 2^S is a set of sets.



Set operations

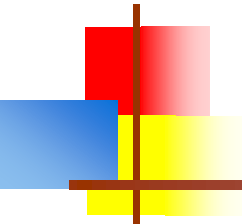
Example 1.1

- If S is the set $\{a, b, c\}$, then its powerset is

$$2^S = \{ \emptyset, \{a\}, \{b\}, \{c\}, \{a,b\}, \{a,c\}, \{b,c\}, \{a,b,c\} \}.$$

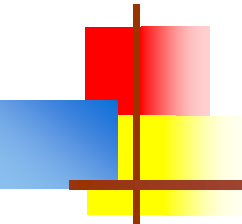
Here $|S| = 3$ and $|2^S| = 2^3 = 8$. This is an instance of a general result; if S is finite, then $|2^S| = 2^{|S|}$.

- If the elements of a set are ordered sequences of elements from other sets, then such sets are said to be the Cartesian product of other sets. The Cartesian product of two sets, which itself is a set of ordered pairs, we write as $S = S_1 \times S_2 = \{ (x,y) : x \in S_1, y \in S_2 \}$



Set operations

- $S_1 \times S_2 \times \dots \times S_n = \{ (x_1, x_2, \dots, x_n) : x_i \in S_i \}$ A set can be divided by separating it into a number of subsets. Suppose that S_1, S_2, S_n are subsets of a given set S and that the following holds:
 1. The subsets $S_1, S_2, \dots, S_n$ are mutually disjoint;
 2. $S_1 \cup S_2 \cup \dots \cup S_n = S$;
 3. none of the S_i is empty. Then $S_1, S_2, \dots, S_n$ is called a partition of S .



Languages and strings

- We are all familiar with the notion of **natural languages**, such as English and French. Still, most of us would probably find it difficult to say exactly what the word “**language**” means.
- *Dictionaries define the term informally as a system suitable for the expression of certain, ideas, facts or concepts, including a set of symbols and rules for their manipulation.*
- While this gives us an intuitive idea of what a language is, it is not sufficient as a definition for the study of formal languages. We need a precise definition for the term.

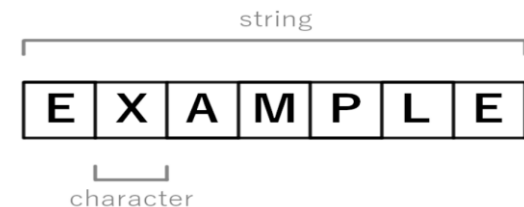
Languages and Strings

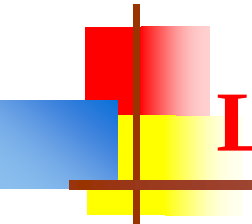
- **Symbol:** any thing like $a, b, c, 0, 1, 2$ etc.
- **Alphabet:** collection of symbols ,it must be finite
 - Alphabet is denoted by Σ
 - Example $\{a, b\}, \{d, e, f, g\}, \{0, 1, 2\}$
- **String :** A string is a finite sequence of symbols drawn from an alphabet
 - A string is always finite in length
 - Example $a, b, 0, 1, ab, ab, ba, bb, 01$
- **Language:** set of strings over a given alphabet
 - Example $\Sigma = \{0, 1\}$

L1 set of all strings of **length 2**

= $\{00, 01, 10, 11\}$

this is finite string

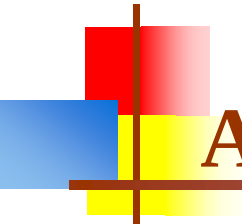




Languages and Strings

Is language finite or infinite?

- A **string** is a finite sequence of symbols drawn from an alphabet. Therefore, a string is always finite in length. However, a language is a set of strings over an alphabet.
- A language can be either finite or infinite, depending on the number of strings it contains. In an **infinite language**, there is no limit to the length of a string, but the length of each string is finite.
- Therefore, while a string is always finite, a language can be either **finite** or **infinite**.
- *No, not all infinite languages can be regular. While some infinite languages can be regular, not all of them can be described by a regular grammar*



Automata

- An **automaton** is an abstract model of a **digital computer**.
- It is the simplest model of computation
- It has a very limited memory
- It has a mechanism for reading input. It will be assumed that the input is a **string** over a given **alphabet**, written on an input file, which the automaton can read but not change.
- The input mechanism can **read** the input file from **left** to **right**, one symbol at a time.
- The input mechanism can also detect the end of the input string (by sensing an end-of-file condition).